Lecture 3

Aashritha Reddy Kukkala - CS23BTECH11030
Nishi Baranwal - CS23BTECH11041

05.08.2024

Memory Operands

Programming languages have simple variables that contain single data elements. They also have complex data structures which can contain more data elements than the registers in a computer. So we'll see how the computer works with them using the example :

C-code :

$$A[12] = h + A[8]$$

Where,

- h is in x21 (means it is already loaded in x21)
- A is an array where each element is of 8 bytes
- Base address of A is in x22

The above is not just a simple addition instruction as we are dealing with arrays and each element of the array has different addresses. So many more instructions have to be executed.

- First, we need to calculate the address of A[8]. Since each element is of 8-Bytes (double-word), the address of A[8] is:
  $x22 + 8 \times 8 = x22 + 64$.
- Then we bring the data of A[8] into a register : load A[8] into a register e.g. in rx
- Then we have to do the addition ry $\leftarrow$ h + rx : where ry is the register where result is stored
- Now we have to calculate the address of A[12]. This will be
  $x22 + 12 \times 8 = x22 + 96$
- Lastly we store the value in ry to A[12].

So here total 5 instructions are required.

Compiled RISC-V Code :

```
addi   x23, x22, 64    // x23 ← x22 + 64
ld   x9, 0(x23)   // x9 ← *(x23)
add   x9, x21, x9   // x9 ← x21 + x9
addi   x23, x22, 96    // x23 ← x22 + 96
sd   x9, 0(x23)    // *(x23) ← x9
```

In the above instructions :

- x23 register is used to store the address of array elements. It is reused for addresses of both the elements.
- ld - load doubleword : loads the data from memory to the register.
  Syntax : ld *destination_register data_memory_address*
- sd - store doubleword : stores the data from register to memory.
  Syntax : sd *source_register data_memory_address*
- We are using the doubleword instructions as we are dealing with 64-bit registers.

Another Variant :

See the following instructions :

```
ld    x9, n(x23)    // x9 ← *(x23 + n)
sd    x9, n(x23)    // *(x23 + n) ← x9
```

The above instructions help us reduce the number of instructions for our code. It accesses the address of the element using the number n (offset) by adding it to the base address (e.g., register x23 in above example). This reduces the extra instructions that we had written previously.

Compiled RISC-V Code :

```
ld    x9, 64(x22)    // x9 ← *(x22 + 64)
add    x9, x21, x9    // x9 ← x21 + x9
sd    x9, 96(x22)    // *(x22 + 96) ← x9
```

- The second method is better than the first method when we know how much offset is required and the number of elements we need to access are less as it decreases the number of steps.

- The first method is better than the second one in cases where the offset is very large as 'n' has a range limit. Even in cases of loops (for or while) with large number of iterations the first method is better.

Eg] C-code :

```
for(i=0; i<500; i++)
  A[i]=1;
```

Where 1 is in x21 and base address of A is in x22.

Compiled RISC-V code :

Method -1 :

```
For loop{
sd    x21, 0(x22)
addi    x22, x22, 8
}
```

Method -2 :

```
sd    x21, 0(x22)
sd    x21, 8(x22)
sd    x21, 16(x22)
...
sd    x21, 3992(x22)
```

- we'll have to write 500s lines of code.

- So by method one though the number of instructions being executed are double than method two, the code is compact.

Another Example :

C-Code :

```
A[12] = h + A[8];
A[10] = k + A[8];
```

h is in x21, k is in x20 and base address of A is in x22.

Compiled RISC-V code :

```
ld    x9, 64(x22)    // x9 ← *(x22 + 64)
add    x10, x21, x9
sd    x10, 96(x22)    // *(x22 + 96) ← x10
add    x10, x20, x9
sd    x10, 80(x22)    // *(x22 + 80) ← x10
```

Above A[8] value is used twice, stored in x9. So, we do not need to load it twice, but only once. This shows that the data elements once loaded to registers can be reused multiple times and forms a very important optimization performed by compilers. The addition and store to memory is very similar to the previous example.

Registers vs Memory

- Since we have a limited number of registers, re-using the values loaded in registers is highly desirable. The code should be optimized to re-use the values.
- We should use registers for variables as much as possible and should only 'spill' to memory for less frequently used variables.

- Register optimisation is important. This job is done by the compiler, currently there are lot of optimisations present in compilers to improve a given code for registers.

Binary Numbers

- Binary numbers are represented in 0s and 1s.
Let X be our binary number then,

$$X = X_{n-1}X_{n-2}\ldots X_0 \text{ for n bit number.}$$

Eg] 101100

Where, X_0 is LSB - Least significant bit
X_{n-1} is MSB - Most significant bit

It's Decimal equivalent is $2^{n-1}X_{n-1} + 2^{n-2}X_{n-2} + \ldots. + 2^0X_0$
So, $101100_2 = 44_{10}$
To prevent confusion we write the number system as subscript to the number.

To convert 44_{10} to binary -

2	44	Remainder
2	22	0
2	11	0
2	5	1
2	2	1
	1	0

So, we keep dividing by 2 till we reach 1 and also note down the remainder for each division. Then our binary number is the sequence of remainders from bottom to top i.e. here it will be 101100.

Negative Binary Numbers

- One way to denote negative numbers is to add an extra bit (MSB) where if it is negative, it is denoted by 1 and if it is positive, it is denoted by 0.
- A disadvantage of adding an extra bit to denote sign is that 0 has two representations (positive and negative) which might lead to complications.
- Another way of representing is using the 2's complement.

2's complement

2's complement representation is adding 1 to 1's complement (invert each of the bits). Ex:

$$X = 1011$$
$$\bar{X} = 0100$$

- Here $\bar{X}$ is 1's complement.

- Adding 1 to $\bar{X}$ gives 0101.
- So 2's complement of X (1011) is 0101.

The 2's complement represents the negative of the number.

Proof :

$$X + \bar{X} = 111...1$$

- Add 1 to this to get 1000..0
- Because the number of bits that we have are limited the leading 1 gets discarded.

$$X + \bar{X} + 1 = 0$$
$$\implies \bar{X} + 1 = -X$$
$$\blacksquare$$

2's complement of 2's complement gives back the original number.

Let Y be the 2's complement of X.

$$Y = 0101$$
$$\bar{Y} = 1010$$
$$\bar{Y} + 1 = 1011$$
$$\blacksquare$$

- This is the same as X.

In 2's complement representation, a value of 1 in the MSB indicates a negative number whereas a value of 0 indicates a positive number. The decimal equivalent of a 2's complement number is:

$$-2^{n-1}X_{n-1} + 2^{n-2}X_{n-2} + \ldots + 2^0X_0$$

- If we represent an 8 bit number in 2's complement system and we want to extend it to 16 bit then we always extend the sign bit. Ex:

$$+4: \quad 0000\ 0100 \rightarrow 0000\ 0000\ 0000\ 0100$$
$$-4: \quad 1111\ 1100 \rightarrow 1111\ 1111\ 1111\ 1100$$

- In RISC-V instruction set:

lb: sign- extend loaded byte
lbu: zero-extend loaded byte

- The above instructions are used to load 8-bit values.
- Depending on if we are using signed or unsigned , the compiler uses the appropriate instruction.
- Range of unsigned is 0 to $2^n - 1$.
- Range of signed is $-2^{(n-1)}$ to $2^{(n-1)}-1$.

Instructions for bitwise operations

Bitwise Manipulations

Operation	C	RISC-V
Shift left	<<	sll,slli
Shift right	>> (>>>, >> *inJava*)	srl,srli,sra,srai
AND	&	and,andi
OR	|	or,ori
XOR	^	xor,xori
NOT	~	xor,xori

- These operations are useful when manipulating at bit level.

- Shift left/right : This logically shifts the number by certain bits

- slli rd, rs1, imm // rd ← rs1 << imm
  Here rd is the destination register, rs1 is the source register and imm is the shift.
- srli rd, rs1, imm // rd ← rs1 >> imm

- In left shift, zeroes are pushed in from LSB's side whereas in right shift zeroes are introduced from MSB's side.

- srai rd,rs1,imm // rd ← rs1 >> imm

- There is a difference between logical and arithmetic shift.
- srai is used to shift to the right arithmetically. In this, we extend the MSB of the source. There is no similar arithmetic shift slai because unlike the MSB, LSB has no special importance.

The register operand versions of these also exist.

- sll rd,rs1,rs2 // rd ← rs1<<rs2 (*logical*)
- srl rd,rs1,rs2 // rd ← rs1>>rs2 (*logical*)
- sra rd,rs1,rs2 // rd ← rs1>>rs2 (*arithmetic*)

- Here the values of rs2 greater than 63 are ignored. Only the last 6 bits of the register rs2 are used. We can't have a shift greater than 63 because a register has only 8 bytes i.e. 64 bits.

- Shift can also used for multiplication or division
- A left shift by n bits is equivalent to multiplying by 2^n.
- A right shift by n bits is equivalent to dividing by 2^n.
- Shifting is cheaper and more efficient than multiplying.
- Ex: Multiplication by 7 can be done by shifting to the left by 3 bits and then subtracting 1.

- AND
and rd,rs1,rs2 // rd ← rs1 & rs2
- This does bitwise AND.

- Using ‘and’ we can selectively make some bits 0.

- OR
or rd,rs1,rs2 // rd ← rs1|rs2
- This does bitwise OR.
- Using ‘or’ we can selectively make some bits 1.

- AND and OR can be combined.
Ex:

```
or x9,x10,x11
and x8,x9,x12
```

- XOR
xor rd,rs1,rs2 // rd ← rs1 ^ rs2
- This performs bitwise XOR.

Performing XOR of a number with a number that has all its digits equal to 1 gives its complement. This is basically a NOT operation. RISC-V uses this to perform a NOT operation. 2’s complement representation of -1 gives the number that has all its digits as 1.

- andi
andi rd,rs1,imm // rd ← rs1 & imm

- ori
ori rd,rs1,imm // rd ← rs1|imm

- xori
xori rd,rs1,imm // rd ← rs1 ^ imm